
V E D I K A H E A L T H

Grievance Window

Implementation Specification

Version 1.0 · April 2026

Platform: Flutter (mobile) + React/TypeScript (web)

Scope: User module · Master Admin module · Backend APIs

Table of Contents

1. Overview

The Grievance Window is a dedicated module within Vedika Health that provides a structured, trackable channel for users to raise complaints, share feedback, and report issues. It replaces ad-hoc communication (phone, email) with a formal system that ensures accountability, transparency, and measurable resolution timelines.

Module Goals
Enable patients, doctors, and vendors to submit structured grievances with supporting evidence.
Give the Master Admin full visibility, triage capability, and a response workflow.
Generate metrics (volume, resolution time, category breakdown) to drive quality improvements.
Trigger real-time notifications to both parties on every status change.

Scope

This document covers the end-to-end implementation of the Grievance Window across all three platforms:

- vedika-health-app — Flutter mobile application (patient-facing)
- vedika-health-web — React / TypeScript web application (patient + admin)
- Backend REST API — to be implemented alongside the frontend modules

Out of Scope (v1.0)

- Third-party ombudsman integration
- Automated resolution via AI triage (planned v2.0)
- Grievance analytics dashboard with charts (planned v2.0)

2. User Flows

2.1 Patient / User Flow

The following steps describe the complete journey a patient takes from discovering the grievance feature to tracking its resolution.

#	Step	Detail
1	Navigate to Grievances	Accessible via bottom navigation bar (mobile) or left sidebar (web). Visible to all verified users.
2	Grievance list screen	Displays all previous grievances as cards with GRV-ID, title, category, date, and colour-coded status badge.
3	Tap 'Raise New Grievance'	Opens the new grievance form. Button is prominent and always visible.
4	Complete the form	User selects category, enters title and description (minimum 50 characters), optionally links an appointment/clinic, sets priority, and attaches up to 5 files.
5	Submit	System creates the grievance with status PENDING. A push notification and in-app alert confirm submission.
6	Track progress	User views the detail screen to see the full conversation thread, status history, and admin replies. Reply box is disabled once status is CLOSED.

Key Screens (User)

- Grievance List Screen — card layout, status badges (red = pending, orange = under review, green = resolved), pull-to-refresh, pagination
- New Grievance Form — category dropdown, title field, description textarea, related-to auto-suggest, file picker (max 5 files, image/PDF only), priority chip selector
- Grievance Detail Screen — chronological message thread (user messages left, admin replies right), status history timeline, add-reply box

2.2 Master Admin Flow

The Master Admin accesses the grievance module from the admin dashboard and manages the complete lifecycle of every grievance raised across the platform.

#	Step	Detail
---	------	--------

1	Dashboard overview	KPI cards show total grievances, pending count, resolved today, and average resolution time.
2	Grievances table	Paginated, searchable table with columns: GRV-ID, user name, category, priority, status, date raised, and action button.
3	Apply filters	Filter by status, date range, category, user type, and priority. Search by GRV-ID or user name.
4	Open detail view	Clicking any row opens a full detail modal: left panel shows user info, grievance content, and attachments; right panel contains admin actions.
5	Mark 'Under Review'	One-click action immediately updates status and sends push notification to the user.
6	Add reply	Admin types a response which appends to the shared conversation thread. Internal remarks field is only visible to admins.
7	Resolve / Close	Resolve sets resolutionDate and resolvedBy. Close prevents further replies. Both trigger a notification to the user.

Key Screens (Admin)

- Grievance Dashboard — KPI summary cards, HIGH priority alert list, recent activity feed
- All Grievances Table — searchable, filterable, paginated; sortable by priority and date
- Grievance Detail Modal — two-panel layout: content + attachments (left), admin reply box + status controls + resolve button (right)

3. Status Lifecycle & Priority

3.1 Status Values

Each grievance progresses through a defined sequence of statuses. Transitions are one-directional with the exception that UNDER_REVIEW can be reopened to PENDING if additional information is needed.

PENDING	Newly submitted. No admin action taken yet.
UNDER_REVIEW	Admin has acknowledged and is investigating.
RESOLVED	Issue addressed. Resolution date logged.
CLOSED	Finalised. No further replies accepted.

3.2 Priority Levels

Priority is set by the user at submission time and can be overridden by the admin. It determines sort order in the admin table and drives SLA targets.

HIGH	Doctor behaviour, billing disputes, safety complaints. Target SLA: < 4 hours.
MEDIUM	General appointment issues, consultation quality. Target SLA: < 24 hours.
LOW	Technical glitches, suggestions, minor inconveniences. Target SLA: < 72 hours.

4. Data Model

4.1 Table: grievances

All fields below map to columns in the primary grievances table. UUID is used as the primary key for all cross-service references.

Field	Type	Description / Values	Required	Default
id	UUID	Primary key, auto-generated	Yes	auto
grievanceId	String	Human-readable ID, format: GRV-YYYYMMDD-NNN	Yes	auto
userId	String	FK referencing the submitting user	Yes	—
userType	Enum	PATIENT DOCTOR VENDOR	Yes	—
title	String	Short summary of the grievance (max 120 chars)	Yes	—
description	Text	Full complaint text. Enforced minimum: 50 chars client-side	Yes	—
category	Enum	See Section 4.2 for allowed values	Yes	—
subCategory	String	Optional free-text sub-topic	No	null
relatedToType	Enum	APPOINTMENT CLINIC DOCTOR APP OTHER	No	null
relatedToId	String	FK to the related entity (appointment ID, clinic ID, etc.)	No	null
attachments	Array<URL>	Storage URLs of uploaded files. Max 5 per grievance	No	[]
status	Enum	PENDING UNDER_REVIEW RESOLVED CLOSED	Yes	PENDING
priority	Enum	LOW MEDIUM HIGH	Yes	MEDIUM
adminRemarks	Text	Internal notes — never returned in user-facing API responses	No	null
responses	JSONB[]	Thread array: { timestamp, message, byAdmin, authorId }	No	[]
resolvedBy	String	Admin user ID — set when status transitions to RESOLVED	No	null
resolutionDate	Timestamp	Auto-set when status transitions to RESOLVED	No	null
createdAt	Timestamp	Auto-set on INSERT	Yes	auto

Field	Type	Description / Values	Required	Default
updatedAt	Timestamp	Auto-updated on every change	Yes	auto

4.2 Category Enum Values

The following categories are available for selection at submission time. This list is returned by the GET /api/grievances/categories endpoint and should be populated dynamically in all dropdowns.

- Appointment Related
- Billing / Payment Issue
- Doctor Behaviour
- Clinic Facility / Hygiene
- Consultation Quality
- Technical / App Issue
- Refund Request
- Others

4.3 Response Thread Schema

Each entry in the responses array follows this structure:

Field	Type	Description
timestamp	Timestamp	UTC timestamp of when the reply was submitted
message	Text	The reply content
byAdmin	Boolean	true if authored by an admin user, false if by the patient
authorId	String	User ID or Admin ID of the author

5. API Specification

All endpoints use JSON (application/json) unless noted as multipart. All user endpoints require a valid Bearer token. Admin endpoints additionally require the MASTER_ADMIN role checked server-side.

5.1 User APIs

Available to authenticated patients, doctors, and vendors. Each user sees only their own grievances.

Method	Endpoint	Description
POST	/api/grievances	Create a new grievance (multipart: JSON fields + file attachments)
GET	/api/grievances	List the current user's grievances. Query params: page, limit, status, category
GET	/api/grievances/:id	Get full detail of a single grievance including the response thread
GET	/api/grievances/categories	Return the list of category enum values for dropdown population

POST /api/grievances — Request Body

Submitted as multipart/form-data to support file attachments alongside JSON fields.

Field	Type	Notes
title	String	Required. Max 120 characters.
description	String	Required. Minimum 50 characters enforced server-side.
category	Enum	Required. Must match a value from /api/grievances/categories.
priority	Enum	Optional. LOW MEDIUM HIGH. Defaults to MEDIUM.
relatedToType	Enum	Optional. APPOINTMENT CLINIC DOCTOR APP OTHER.
relatedToId	String	Optional. Required if relatedToType is provided.
attachments	File[]	Optional. Max 5 files. Accepted: image/png, image/jpeg, application/pdf. Max 5 MB each.

5.2 Master Admin APIs

All endpoints below require the MASTER_ADMIN role. Attempts from non-admin tokens must return 403 Forbidden.

Method	Endpoint	Description
GET	/api/admin/grievances	List all grievances across all users. Supports filters: status, dateFrom, dateTo, category, userType, priority, search
GET	/api/admin/grievances/:id	Full detail view including adminRemarks and full response thread
PUT	/api/admin/grievances/:id/status	Update status and optionally add a reply to the thread in a single atomic call
POST	/api/admin/grievances/:id/reply	Append a reply to the conversation thread without changing status
GET	/api/admin/grievances/stats	KPI summary: total, pending, resolvedToday, avgResolutionHours, countByCategory

PUT /api/admin/grievances/:id/status — Request Body

Field	Type	Notes
status	Enum	Required. New status value. Must be a valid forward transition.
reply	String	Optional. If provided, appended to responses[] as a byAdmin: true entry.
adminRemarks	String	Optional. Internal-only note. Never returned to users.

5.3 Notification Triggers

The notification service must fire on every status transition. Both push (FCM/APNs) and in-app notifications are required.

Event	Notify	Message
Grievance submitted	User (confirmation)	<i>Your grievance {GRV-ID} has been received.</i>
Grievance submitted	Admins (new ticket)	<i>New {priority} grievance received: {title}</i>
Status → UNDER_REVIEW	User	<i>Your grievance {GRV-ID} is being reviewed.</i>

Admin reply added	User	<i>Admin responded to your grievance {GRV-ID}.</i>
Status → RESOLVED	User	<i>Your grievance {GRV-ID} has been resolved.</i>
Status → CLOSED	User	<i>Your grievance {GRV-ID} has been closed.</i>

6. Frontend Design Specifications

6.1 User Side (App + Web)

Grievance List Screen

- Card layout — one card per grievance
- Each card shows: GRV-ID, title (truncated at 60 chars), category tag, submission date, and status badge
- Status badge colours: red (#E24B4A) = PENDING, orange (#EF9F27) = UNDER_REVIEW, green (#3B6D11) = RESOLVED, grey = CLOSED
- Pull-to-refresh on mobile; infinite scroll or pagination (10 per page) on web
- 'Raise New Grievance' floating action button (mobile) or header button (web) — always visible

New Grievance Form

- Category — required dropdown, values from GET /api/grievances/categories
- Title — text field, max 120 characters, character counter visible
- Description — textarea, minimum 50 characters enforced on submit with inline error
- Related to — optional auto-suggest field pre-populated with the user's recent appointments
- File upload — native file picker, image/PNG, image/JPEG, PDF only, max 5 MB per file, max 5 files
- Priority — chip selector (Low / Medium / High), default Medium
- Submit button — large, full-width on mobile, disabled until required fields are valid

Grievance Detail Screen

- Chronological message thread: user messages aligned left, admin replies aligned right with a blue left border accent
- Each message shows: author label, timestamp, message text
- Status change events appear as system messages in the thread (e.g. 'Status changed to UNDER_REVIEW — 21 Apr 2026')
- Reply text input + Send button visible at the bottom; hidden (not just disabled) when status = CLOSED

6.2 Master Admin Side (Web)

Grievance Dashboard

- KPI summary cards in a 4-column row: Total, Pending, Resolved Today, Avg Resolution Time (hrs)

- Alert list below the cards: top 5 HIGH priority unresolved grievances with direct action links
- Recent activity feed showing the last 10 status changes across all grievances

All Grievances Table

- Columns: GRV-ID, User Name, User Type, Category, Priority, Status, Date Raised, Action
- Priority column renders colour-coded pills (same colours as status badges)
- Filters: status (multi-select chips), date range picker, category dropdown, user type, priority, free-text search by GRV-ID or user name
- Default sort: priority (HIGH first), then date (oldest first)
- Pagination: 20 rows per page

Grievance Detail Modal

- Full-screen overlay on desktop, bottom sheet on tablet
- Left panel (60% width): user profile card (name, type, contact), grievance content, attachments as thumbnails, full conversation thread
- Right panel (40% width): status dropdown, internal remarks textarea, reply text area with Send button, Resolve button (green), Close button (grey)
- Resolving while leaving a reply in the text area should automatically append the reply before resolving (UX shortcut)

7. Implementation Plan

Implementation follows a backend-first approach to unblock both frontend teams in parallel. Each phase has a defined checklist and acceptance criteria.

Phase 1 — Backend (Week 1–2)	
☐	Create Grievance model and run database migration
☐	Implement auto-generation of human-readable GRV-ID on INSERT
☐	Implement all 4 user-facing REST endpoints with proper auth middleware
☐	Implement all 5 admin REST endpoints with MASTER_ADMIN role guard
☐	Multipart file upload support (reuse clinic registration pattern); store to cloud storage
☐	Implement notification service triggers for all 6 events (Section 5.3)
☐	Write unit tests for status transition validation (reject invalid transitions server-side)
☐	API documentation (Swagger / Postman collection) shared with frontend teams

Phase 2 — User Frontend: Flutter App (Week 2–3)	
☐	Add 'Grievances' tab to bottom navigation bar
☐	Implement Grievance List screen with card layout and status badges
☐	Implement pull-to-refresh and infinite scroll/pagination
☐	Implement New Grievance Form with all fields and client-side validation
☐	Implement file picker with type/size restrictions and preview
☐	Implement Grievance Detail screen with conversation thread
☐	Integrate all 4 user APIs
☐	Handle push notification deep links to the correct grievance detail screen

Phase 3 — User Frontend: React Web (Week 2–3, parallel)	
☐	Add 'Grievances' entry to the left sidebar with unread badge count
☐	Implement Grievance List page (mirrors Flutter screen, adapted for web layout)

☐	Implement New Grievance Form as a page (not modal) for accessibility
☐	Implement Grievance Detail page with responsive thread layout
☐	Integrate all 4 user APIs
☐	In-app notification banner on status changes (WebSocket or polling fallback)

Phase 4 — Admin Frontend: React Web (Week 3–4)

☐	Create Grievance Management section in admin panel navigation
☐	Implement Grievance Dashboard with KPI cards and HIGH priority alert list
☐	Implement All Grievances Table with filters, search, and default sort
☐	Implement Grievance Detail Modal with two-panel layout
☐	Implement reply box, status dropdown, internal remarks, Resolve and Close buttons
☐	Integrate all 5 admin APIs

Phase 5 — Testing & Polish (Week 4–5)

☐	End-to-end status transition flow testing (PENDING → UNDER_REVIEW → RESOLVED → CLOSED)
☐	Role-based access: verify non-admin tokens receive 403 on all /api/admin/* routes
☐	File upload: test type enforcement, size limits, and CDN URL storage
☐	Push notification delivery testing on iOS and Android
☐	Validate adminRemarks is never returned in user-facing API responses
☐	Accessibility review: form labels, keyboard navigation, screen reader support
☐	Performance: table pagination, list rendering with 500+ records
☐	Cross-browser testing: Chrome, Safari, Firefox, Edge

8. Validation Rules

All validation below is enforced both client-side (for UX) and server-side (for security). Client-side validation must never be the sole guard.

Field	Rule	Error Message
title	Required, max 120 chars	<i>Title is required and must be under 120 characters.</i>
description	Required, min 50 chars	<i>Please describe your grievance in at least 50 characters.</i>
category	Required, must match enum	<i>Please select a valid category.</i>
relatedToId	Required if relatedToType set	<i>Please specify the related appointment or clinic.</i>
attachments	Max 5 files, max 5 MB each	<i>You can attach up to 5 files (images or PDFs), each up to 5 MB.</i>
attachments	Type: image/png, image/jpeg, pdf	<i>Only PNG, JPEG, and PDF files are accepted.</i>
status (PUT)	Valid forward transition only	<i>This status transition is not permitted.</i>

9. Next Steps & Prompts

The following ready-to-use prompts can be passed directly to Cursor, Antigravity, or any AI coding assistant to generate implementation artefacts.

Generate: Backend Model + Migration

Create the Grievances database model and migration based on the schema in Section 4, including indexes on `userId`, `status`, `priority`, and `createdAt`.

Generate: All REST API Controllers

Implement all 9 endpoints from Section 5 using the existing Vedika Health API patterns. Include role-based middleware, multipart file handling, and notification triggers.

Generate: Flutter Screens

Build the three Flutter screens (Grievance List, New Grievance Form, Grievance Detail) following the design specs in Section 6.1. Use the existing design system and integrate the 4 user APIs.

Generate: Admin React Components

Build the Grievance Dashboard, All Grievances Table, and Grievance Detail Modal in React/TypeScript following Section 6.2. Integrate all 5 admin APIs.

Document prepared by: **Vedika Health Engineering** | Last updated: April 2026

This document is confidential and intended for internal use only.